CS 440 Fall 2017
Assignment 4: Perceptrons and Reinforcement Learning

Abhinav Kohar – 4 credits
Adam J. Stewart – 4 credits

December 11, 2017

Part 1: Digit Classification using Discriminative Machine Learning Methods

Part 1.1: Digit Classification with Perceptrons

Implementation

Our implementation of the perceptron classifier uses the standard perceptron training algorithm covered in lecture. First, we read in the training data, testing data, training labels, and testing labels. We keep a weight vector for each digit class with 28x28 features. The weight vector is multiplied by the feature vector for each class, and the maximum resulting value is selected as a class label. This weight vector is updated over subsequent epochs through training data, based on the prediction of the classifier. With the multi-class perceptron, we update only when the predicted label is different from the actual label. The classifier for the actual label is increased in weight and classifier for the predicted label is decreased in weight. To incorporate bias we add a value of 1 when initializing each feature vector (training and test data) and weight vectors. Results of different tuning parameters will be discussed subsequently.

Accuracy, training curve, and confusion matrix for different parameter settings

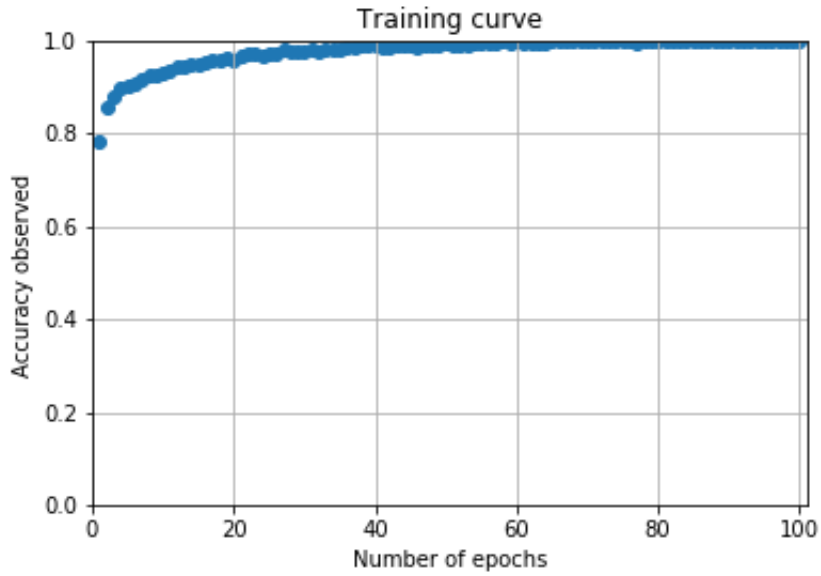
The different parameters we used were bias, learning decay rate, weight initialization, ordering of examples, and number of epochs, as suggested in the assignment.

100 epochs have been used in all the experiments. The learning rate was calculated according to the following formula:

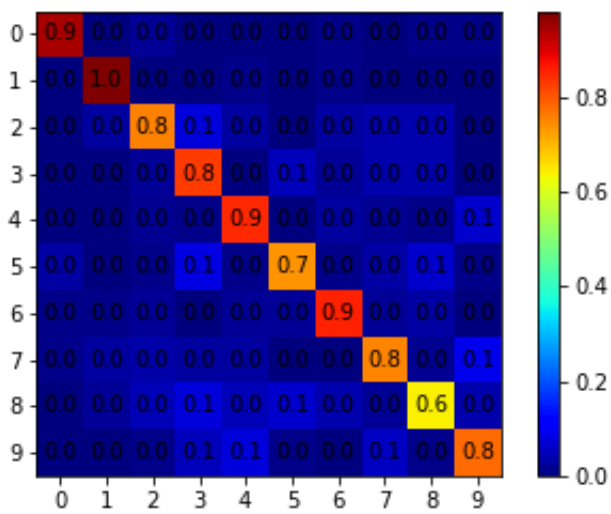
$$\alpha(epoch) = \frac{C_\alpha}{C_\alpha + epoch} \tag{1}$$

Only 100 epochs were required since the perceptrons learned all training examples with full accuracy with this number of epochs. We tried a learning decay rate value $C_\alpha = 100$, and attained an overall accuracy of 81%. A learning decay rate of $C_\alpha = 150$ gives better accuracy and has been used in all the experiments below. (The raw data - confusion matrix, exact values accuracy as function of epochs - is generated by the program as well, we are just presenting charts here.)
Training with no bias, random initial weights, and reordering of samples gives best overall accuracy of: 82.6%.

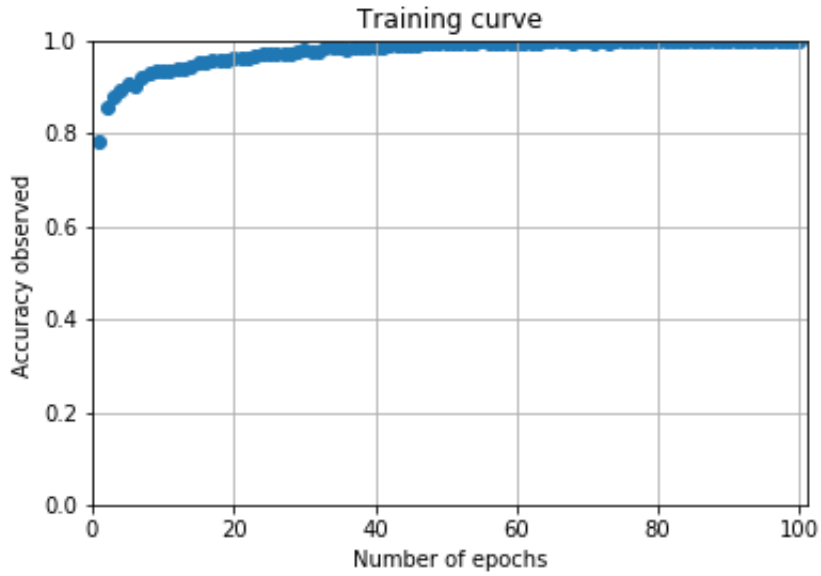
1. Training with bias, zero initial weights, and no reordering of samples
Overall accuracy: 81.3%
Training curve:



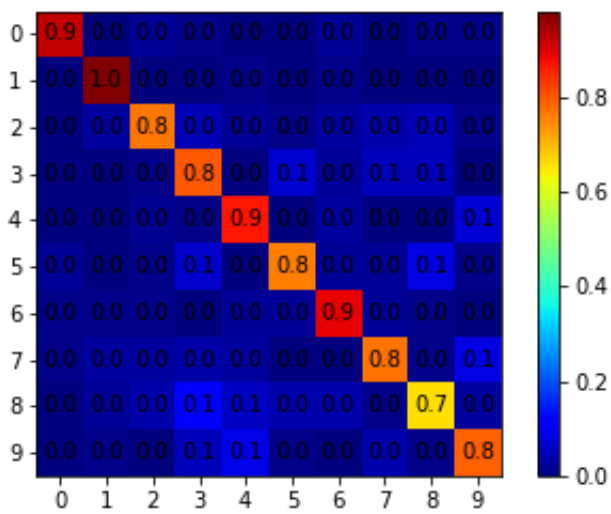
Confusion matrix:



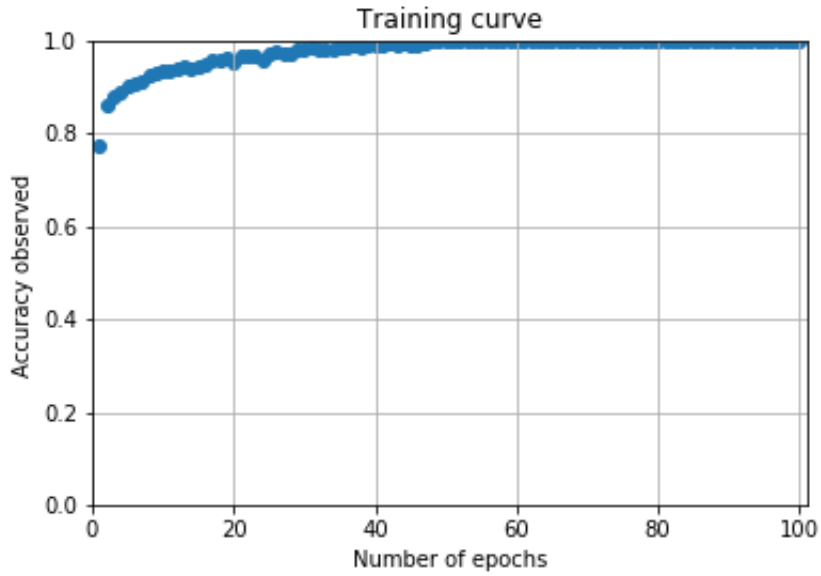
2. Training with bias, random initial weights, and no reordering of samples
Overall accuracy: 82.1%
Training curve:



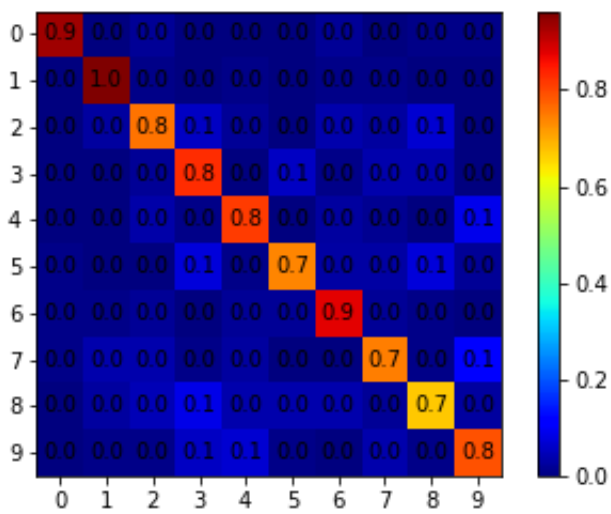
Confusion matrix:



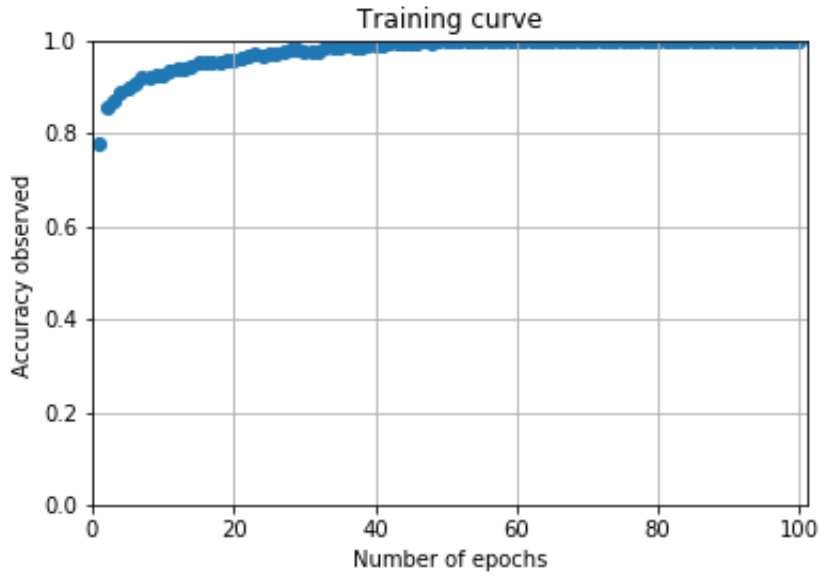
3. Training with bias, random initial weights, and reordering of samples
Overall accuracy: 81.1%
Training curve:



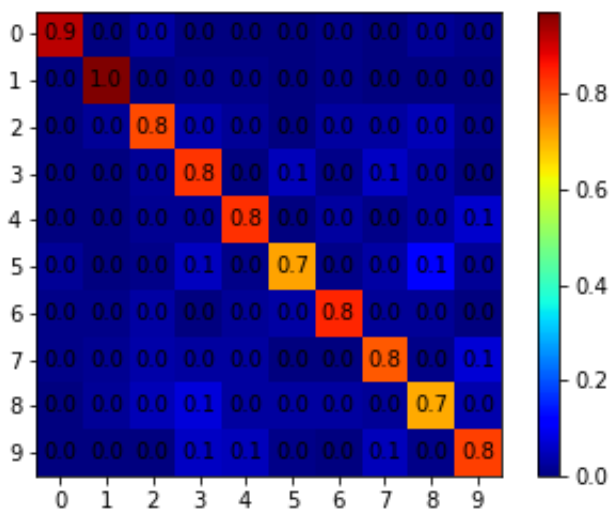
Confusion matrix:



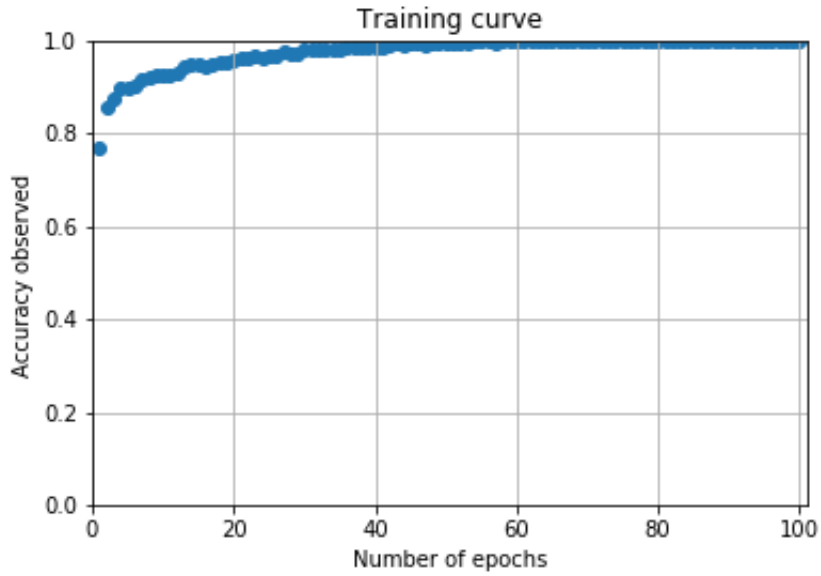
4. Training with bias, zero initial weights, and reordering of samples
Overall accuracy: 82.5%
Training curve:



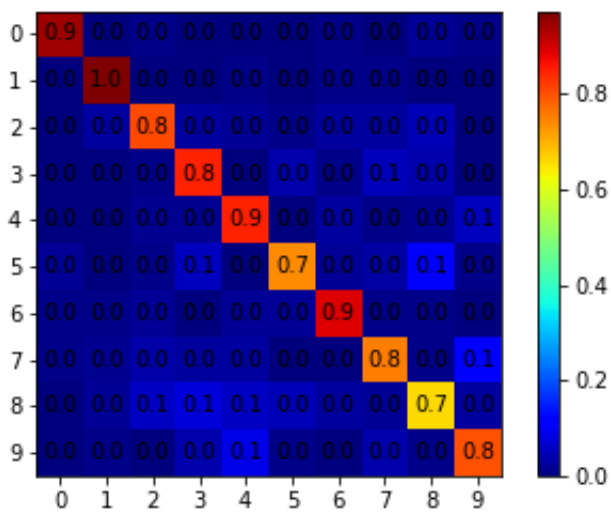
Confusion matrix:



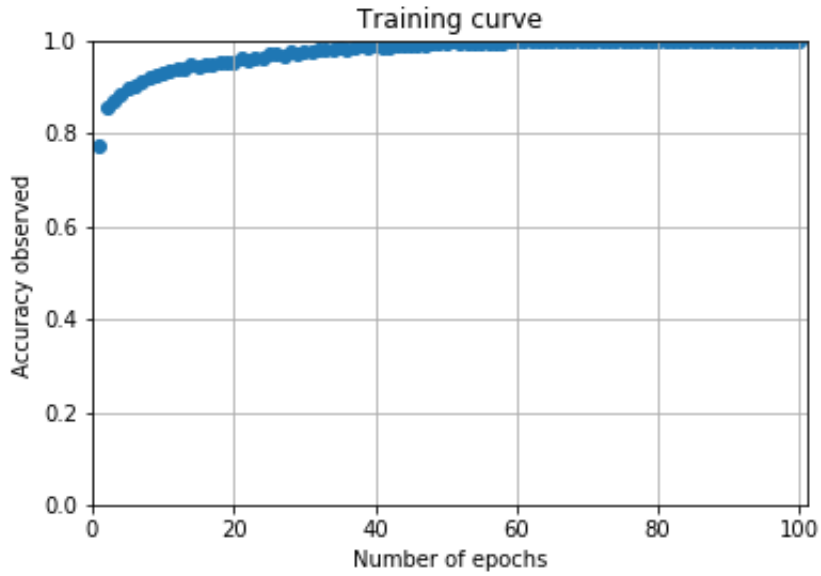
5. Training with no bias, random initial weights, and reordering of samples
Overall accuracy: 82.6%
Training curve:



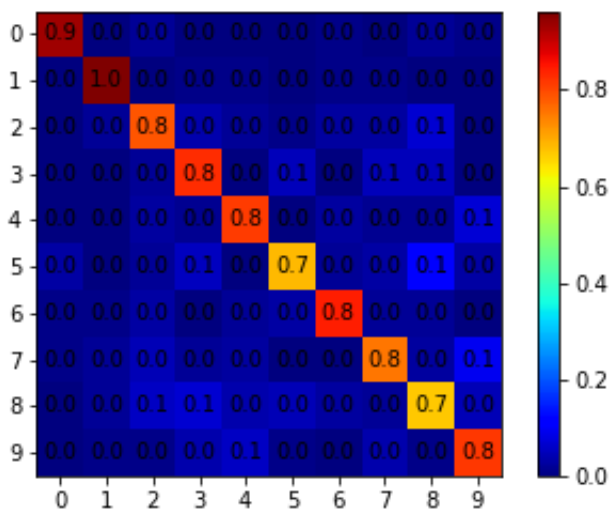
Confusion matrix:



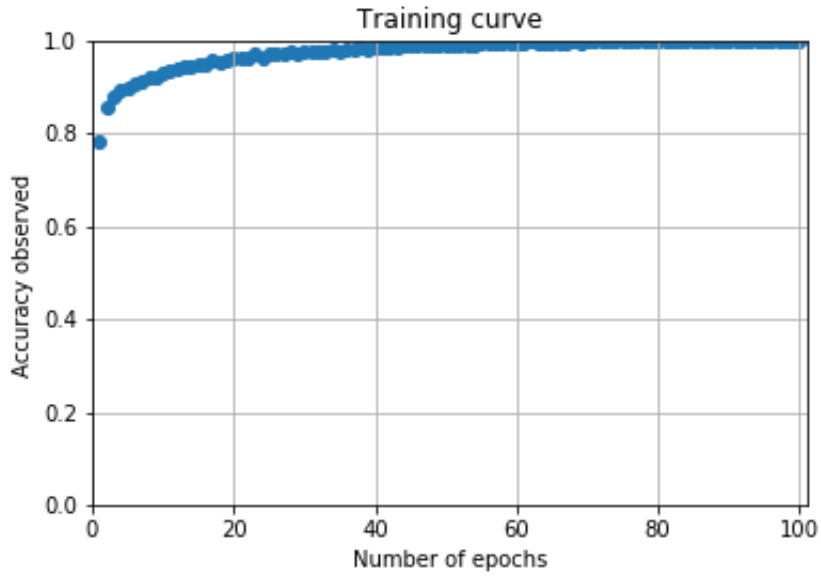
6. Training with no bias, zero initial weights, and reordering of samples
Overall accuracy: 81.0%
Training curve:



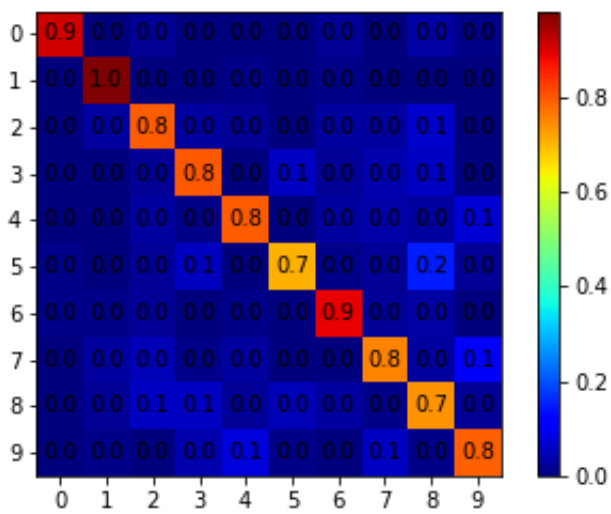
Confusion matrix:



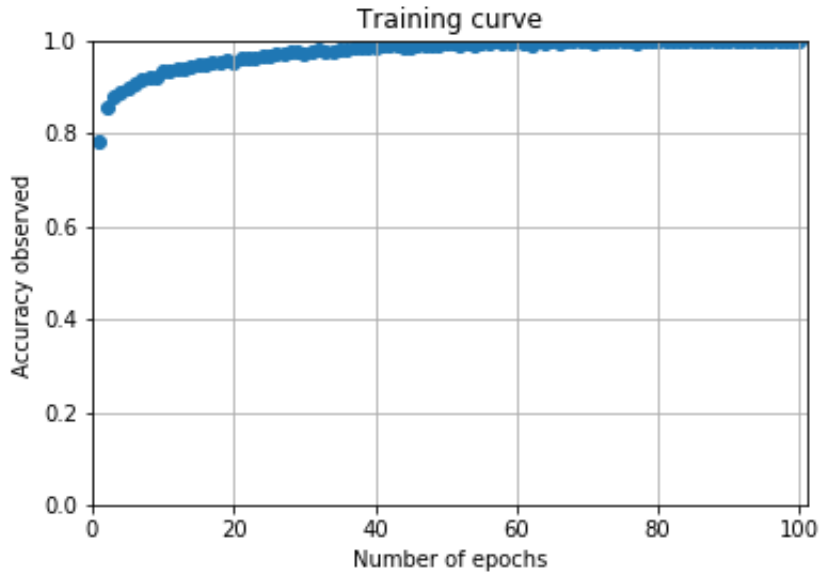
7. Training with no bias, zero initial weights, and no reordering of samples
Overall accuracy: 81.6%
Training curve:



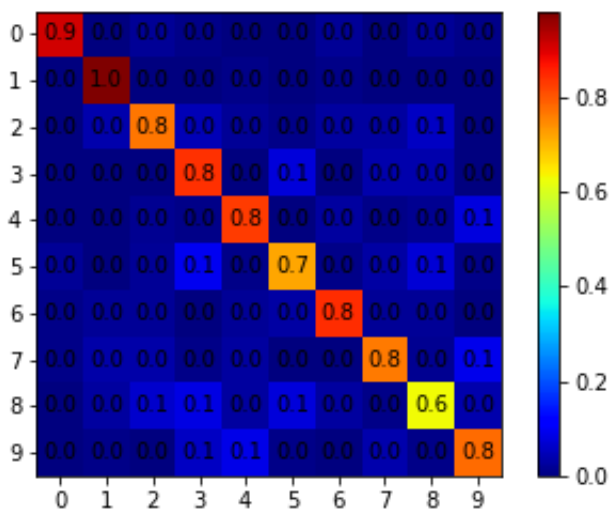
Confusion matrix:



8. Training with no bias, random initial weights, and no reordering of samples
Overall accuracy: 80.7%
Training curve:



Confusion matrix:



Comparison with vanilla Naïve Bayes

From Assignment 3, our vanilla Naïve Bayes implementation gave an overall classification accuracy of 77.40%. In comparison, our new perceptrons classifier gave an overall accuracy of 82.6%. One reason why perceptrons do better than Naïve Bayes is that Naïve Bayes makes a assumption of conditional independence, while the perceptron classifier tries to model the relationship between different features. Also, digit wise accuracy for each class with the perceptron classifier is better than with Naïve Bayes.

Part 1.2: Digit Classification with Nearest Neighbor

Implementation

We implemented the K-nearest neighbors algorithm, where you compute the distance of each feature in each testing data image with each feature in the training data images. We found the top K neighbors and did a majority vote to label the given test data.

We used Hamming distance as a similarity measure between the images. In both Assignment 3 and Assignment 4, we do a mapping from input image features to binary codes. Hamming distance is a natural similarity measure on binary codes or categorical variables, and can be computed with just a few machine instructions per comparison.

Hamming distance is defined as follows: if two feature values are the same, the distance between them is 0, while if they differ, the distance is 1. The total distance between two images for our case is the sum of all the 1's.

feature 1	feature 2	Hamming distance
0	0	0
0	1	1
1	0	1
1	1	0

Table 1: Hamming distance between different kinds of features

Algebraically, we define the Hamming distance as follows:

$$D_H = \sum_{i=1}^{k} |x_i - y_i|$$
$$x = y \Rightarrow D = 0$$
$$x \neq y \Rightarrow D = 1$$

Overall accuracy on test set as a function of k: (Exact values in .txt files outputted by program)

Best accuracy achieved for $k = 3$ was 90.4%

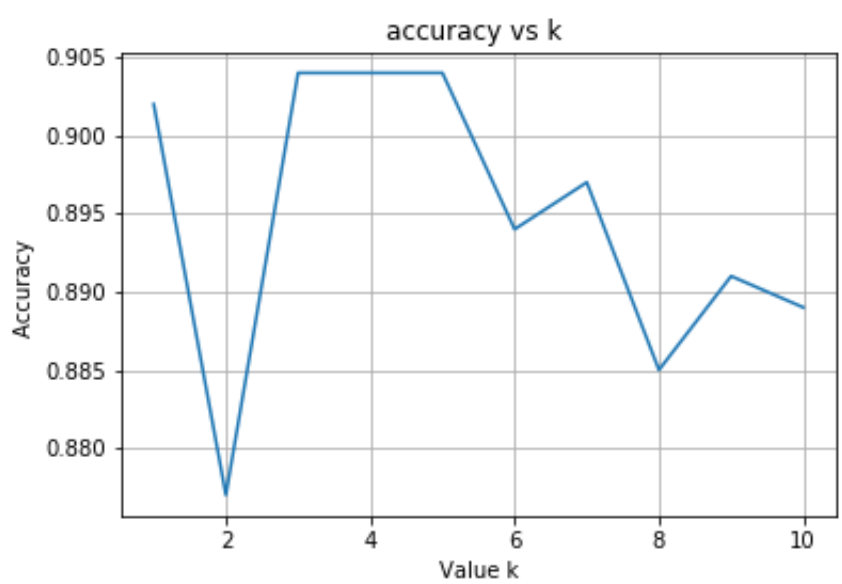


Figure 1: Accuracy of classifier for different values of k (zoomed in y range to see trends better)

Confusion matrix for best choice of $k = 3$: (Exact values in .txt files outputted by program)

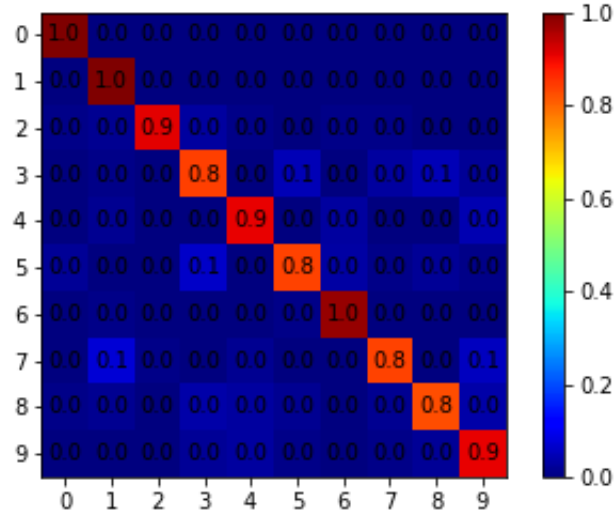


Figure 2: Confusion matrix for best choice of $k = 3$

Running times for different values of k

There is no training time for nearest neighbor search, but testing requires a lot of computation. Run times vary in the range of 66.7 seconds to 68 seconds. This is because we always compute the distance between each test point and all the training data, no matter the value of k. The k nearest neighbors algorithm is $O(n^2)$, making it independent of k. Picking k nearest neighbours and majority voting are $O(1)$ tasks after sorting the data based on distance. The slight variations can be accounted by OS process scheduling and context switching.

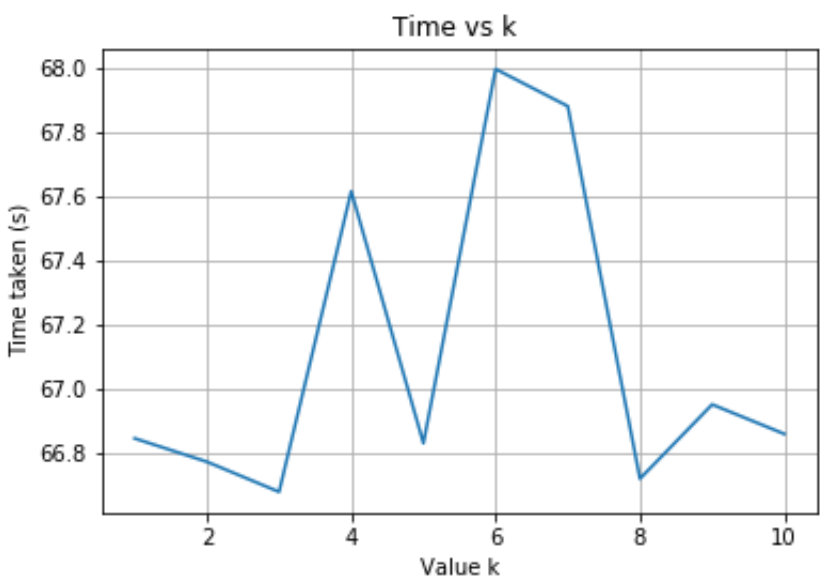


Figure 3: Run time in seconds for different values of k (zoomed in y range to see trends better)

Comparison of k-Nearest Neighbour with Naïve Bayes and Perceptron classifier

k-Nearest Neighbour (kNN) with $k = 3$ performs better than both the Naïve Bayes and Perceptron classifiers. The best accuracy achieved is 90.4% compared to 77.40% and 82.60% for the Naïve Bayes and Perceptron classifiers, respectively. Although kNN has higher accuracy and no training time, it takes more time to classify each test image.

Part 1: Extra Credit

Advanced features

We have incorporated advanced ternary features from assignment 3 into our perceptron classifier. The feature vector was given weights based on the three features: the weight was 0 if the feature was ' ', the weight was 1 if the feature was '+', and the weight was 2 if the feature was '#'. An overall accuracy of 82.7% was achieved using ternary features. It is a small improvement over our best perceptron classifier with binary features, which had 82.6% accuracy. We also tried all possible parameter combinations as stated in part 1.1. We are reporting our best results. Below, we show the training curve and confusion matrix for this classifier.

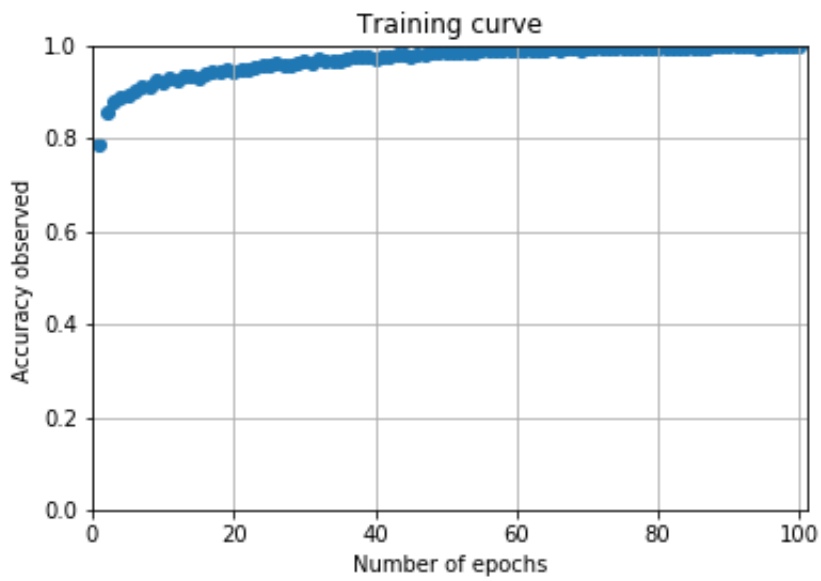


Figure 4: Training curve ternary features, perceptron classifier (with no bias, randomly initialized weights and no reordering of training samples)

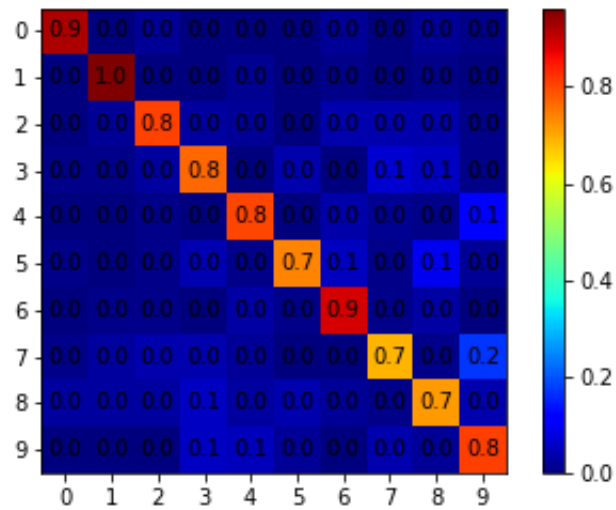


Figure 5: Confusion matrix of ternary features, perceptron classifier (with no bias, randomly initialized weights and no reordering of training samples)

Visualizing the learned perceptron weights

We visualized the perceptron weights as 28x28 images. We are using digit class 0 and digit class 1 as examples here. We will present weight images for perceptron classifier with no bias, randomly initialized weights and reordered training samples.

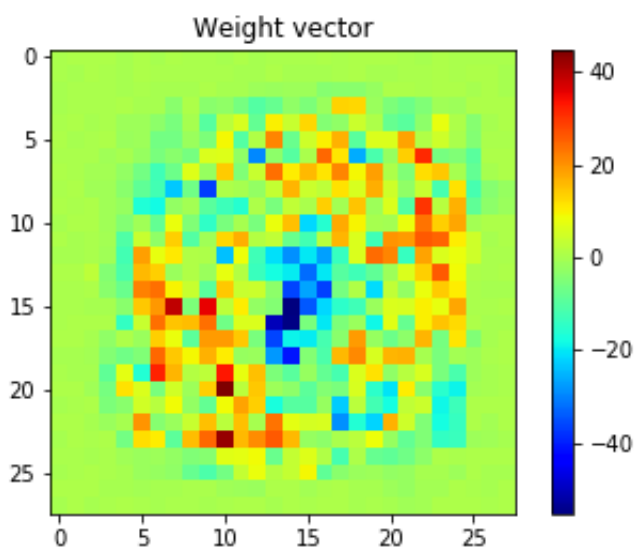


Figure 6: Weights for digit class 0 of a perceptron classifier (with no bias, randomly initialized weights and reordered training samples)

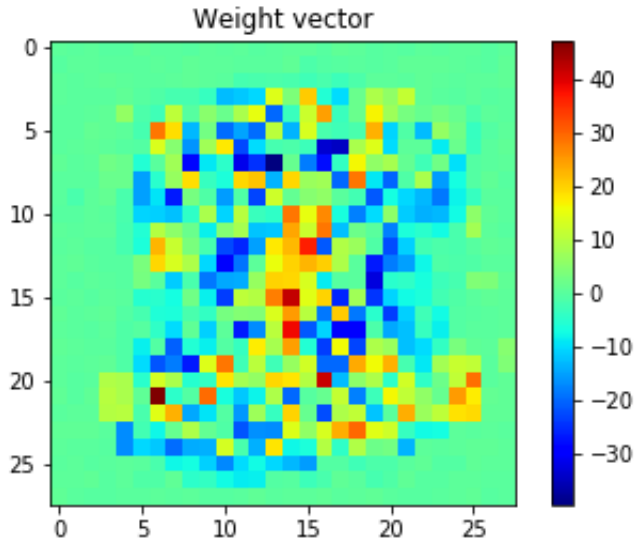


Figure 7: Weights for digit class 1 of a perceptron classifier (with no bias, randomly initialized weights and reordered training samples)

In the given images, we use 'jet' as our color mapping. Blue represents negative numbers, red represents positive numbers, and green is neutral where nothing is present. Observing these images, we see that for digit 0, the positive weight forms a cycle in the image, which shows the general pattern of digit 0 while the digit 1 only has positive weight in the middle column. For 0, the center area is occupied by negative weights and for 1 the boundary area of 1 is occupied by negative weights. Therefore, in general, the positive weight represents where the real digit pattern should be and negative weight represents misclassification. If a feature has value in the positive weight area, it will contribute to the classification. On the other hand, a value in the negative weight area will counter the classification result.

Differentiable perceptron learning rule

We implemented the delta learning rule, with sigmoid (differential) activation. The delta rule is a gradient descent learning rule for updating the weights of the inputs. For a neuron j with activation function $g(x)$ (here sigmoid) the delta rule for j's i'th weight is given by:

$$\Delta w_{ji} = \alpha(t_j - y_j)g'(h_j)x_i$$

where α is the learning rate,
$g(x)$ is the activation function,
t_j is the target output,
h_j is the weighted sum of neuron inputs,
y_j is the actual output,
x_i is the i'th input.

The overall accuracy we achieved is 81.1%. We present the training curve and confusion matrix as previously seen. The accuracy is similar to the non-differentiable classifier. We did not tune too many parameters for this one, otherwise it might have been slightly better. But intuitively they will be close in accuracy.

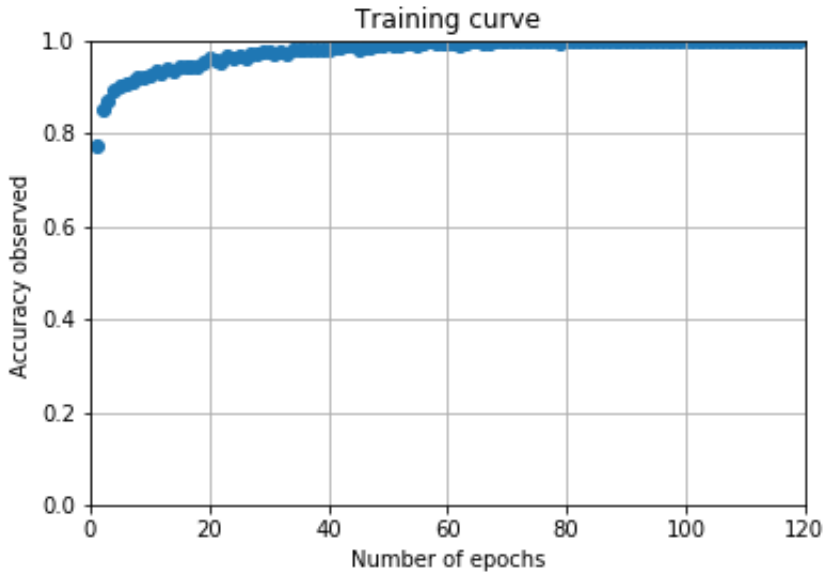


Figure 8: Training curve differentiable perceptron classifier (with no bias, randomly initialized weights and re-ordered of training samples)

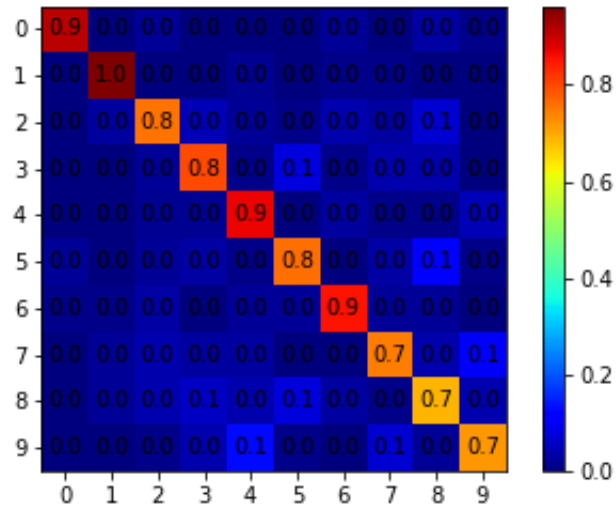


Figure 9: Confusion matrix differentiable perceptron classifier (with no bias, randomly initialized weights and reordered of training samples)

SVM classifier for digit data

We trained the builtin SVM classifier from `sklearn` on the digit data. We did an exhaustive search over the specified parameter values for an estimator. We used `GridSearchCV`, which implements a "fit" and a "score" method. It also implements "predict", "predict_proba", "decision_function", "transform" and "inverse_transform" methods, since SVM implements them. The parameters of the estimator used to apply these methods are optimized by cross-validated grid-search over a parameter grid.

We achieved an overall accuracy of 95.9% for the test data set with this exhaustive search.

We show the heat map of our two tuned parameters, γ and C. We use the 'auto' color map. The interesting search happens from green to light blue parts in this color map.

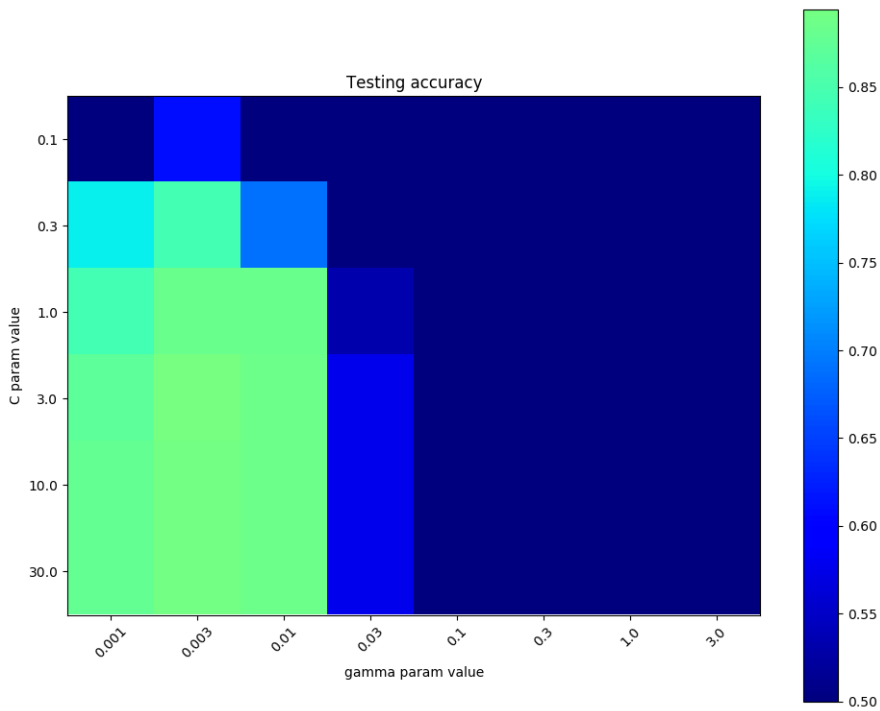


Figure 10: Heat map of parameters, C and gamma

Now we present our confusion matrix:

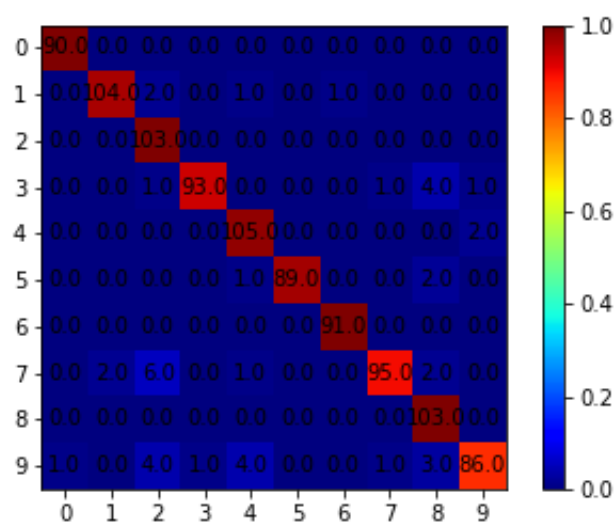


Figure 11: Confusion matrix SVM on digits data

Part 2: Q-learning (Pong)

Part 2.1: Single-Player Pong

Implementation

In our implementation of Pong, we used a Model-Controller-View (MVC) software architecture. In this architecture, all user interactions are managed by the Controller. The Controller then manipulates the Model, which handles all in-game physics. The Model then updates the View, which is responsible for drawing the GUI that the user sees. This paradigm is represented in the following diagram:

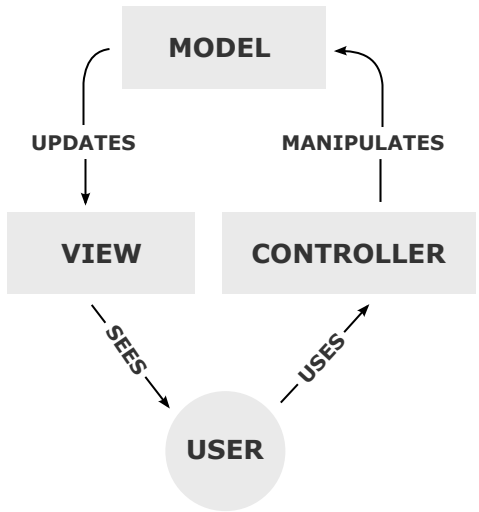


Figure 12: Diagram of interactions within the MVC pattern

In our program, this involved the following file hierarchy:

```
pong/__main__.py
pong/constants.py
pong/controller.py
pong/model.py
pong/view.py
```

`__main__.py` was responsible for launching the program, and handled all command-line argument parsing. Launching a new game is as simple as:

```
python pong --player2=agent
```

`constants.py` held all the constants used throughout the program, such as the dimensions, velocities, and colors of the overall game, ball, and paddles. It also stored the hyperparameters for the learning rate, discount factor, and exploration function, as well as the variables used to discretize the state of the game.

`controller.py` contained a class for the game itself, which allowed the Player class to interact with the State class. Each Player is responsible for its own paddle. `controller.py` contains an outer loop in the program that loops through each training and testing game, and an inner loop that loops until the ball goes out of bounds. This inner loop is responsible for updating the Model as well as drawing the View.

`model.py` contained several classes for managing the State of the game, the Ball, and the Paddle. Each Ball and Paddle instance is responsible for motion and detection of collisions and bounces. These classes ensure that the paddle does not go out of bounds, and that the ball velocities are randomized after a collision with the paddle. They are also responsible for discretizing the state of the game at any given time.

`view.py` was the simplest of all of the files. It was responsible for drawing the current state of the game at each frame. It primarily served as a wrapper around the `pygame` module, which handled most of the work involved in the GUI.

Hyperparameters

In our Q-learning implementation, we used the following formula to update our Q-values at each time step:

$$Q(s_t, a_t) \leftarrow (1-\alpha) \cdot Q(s_t, a_t) + \alpha \cdot (r_t + \gamma \cdot \max_a Q(s_{t+1}, a)) \tag{2}$$

This formula has a couple hyperparameters that control the convergence of our Q-learning implementation, namely the **learning rate** α and the **discount factor** γ.

Our learning rate α was calculated using the following formula:

$$\alpha(t) = \frac{C_\alpha}{C_\alpha - 1 + t} \tag{3}$$

In this formula, C_α is the α decay rate and t is the time-step (number of games). In order to optimize our α decay rate, we performed a grid search over possible values of $C_\alpha \in [500, 4000]$. We found the optimal α decay rate to be $C_\alpha = 1000$.

Our discount factor γ was held constant. In order to optimize γ, we performed a grid search over possible values of $\gamma \in [0.1, 0.9]$. We found the optimal discount factor to be $\gamma = 0.5$.

Our exploration function used ϵ-**greedy** to determine whether to explore (choose a random action) or exploit (choose the optimal action). Our value of ϵ was calculated using the following formula:

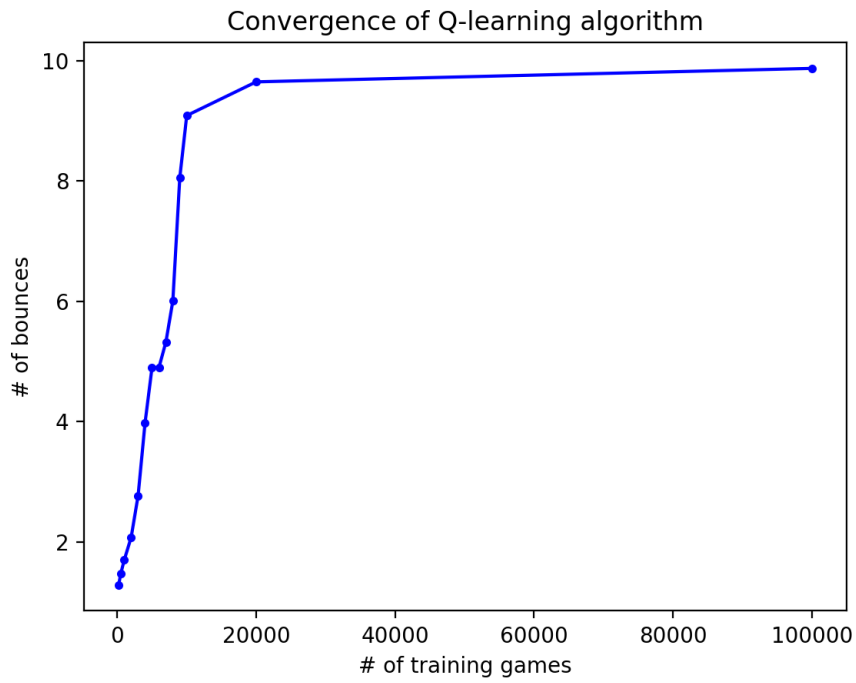
$$\epsilon(t) = \frac{C_\epsilon}{C_\epsilon - 1 + t} \tag{4}$$

In this formula, C_ϵ is the ϵ decay rate and t is the time-step (number of games). Using this formula, we can initially start off with more exploration and transition to more exploitation towards the end. In order to optimize our ϵ decay rate, we performed a grid search over possible values of $\epsilon \in [50, 1000]$. We found the optimal ϵ decay rate to be $C_\epsilon = 300$.

Performance

Using the aforementioned optimized hyperparameters, we attained the following results. With 10,000 training games, we were able to attain a bounce rate of 9.091 bounces averaged over 1,000 testing games. With 100,000 training games, we were able to attain a bounce rate of 9.876 bounces averaged over 1,000 testing games.

In the following figure, you can see the convergence rate of the algorithm:



As you can see, it takes around 10,000 games for the agent to learn an optimal policy.

Modified MDP

Our initial idea for improving the bounce rate of our Q-learning agent was to modify the discretization values. Since the paddle can only move a distance of 0.04 at a time and ranges over a position from 0 to 0.8, we changed the paddle discretization function to return a value in the range 0 to 20. Similarly, we changed the ball discretization to a 20x20 grid.

We re-optimized our hyperparameters for this new MDP. We found the new optimum α decay rate to be $C_\alpha = 6000$ after a grid search over values of $C_\alpha \in [500, 16000]$. We found the new optimum discount factor to be $\gamma = 0.8$ after a grid search over values of $\gamma \in [0.1, 0.9]$. We found a new optimum ϵ decay rate to be $C_\epsilon = 25$ after a grid search over values of $C_\epsilon \in [10, 600]$.

Initial results showed horrible performance. With 10,000 training games, we were able to attain a bounce rate of only 3.512 bounces averaged over 1,000 testing games. With 100,000 training games, we were able to attain a bounce rate of only 6.09 averaged over 1,000 testing games. It is possible that a larger number of training games would result in better performance over the previous discretization variables, but it was not computationally feasible to test this idea given the due date of the assignment.

Our second idea for improving the bounce rate of our Q-learning agent was to modify the reward model. Under the previous system, the penalty for losing the ball wasn't very strong. The Q-learning agent was attempting to learn a strategy where bouncing the ball was just as important as avoiding losing the ball. However, that is not the case. If you bounce the ball, you have a chance to bounce the ball again and again in order to improve your score. If you lose the ball, that game is over and you won't have any more chances. In that sense, losing the ball is much worse than bouncing the ball is good.

With that in mind, we tried out various values for rewards and penalties. Any values such that $|reward| > |penalty|$ showed terrible performance. Values such that $|penalty| > |reward|$ were much more successful. After performing a grid search over possible penalties in the range $[-1, -100]$, we found an optimal penalty of -10. We kept the reward at 1 point for each bounce.

We re-optimized our hyperparameters for this new reward model using the old discretization scheme. We found the new optimum α decay rate to be $C_\alpha = 1200$ after a grid search over values of $C_\alpha \in [250, 4000]$. We

found the new optimum discount factor to be $\gamma = 0.8$ after a grid search over values of $\gamma \in [0.1, 0.9]$. We found the new optimum ϵ decay rate to be $C_\epsilon = 300$ after a grid search over values of $\epsilon \in [150, 600]$.

The new reward model proved to be significantly better. With 10,000 training games, we were able to attain a bounce rate of 12.836 averaged over 1,000 testing games. With 100,000 training games, we were able to attain a bounce rate of 16.951 averaged over 1,000 testing games.

Part 2.2: Two-Player Pong

Implementation

In order to allow the game to have two players with different strategies, we implemented a Player abstract base class with subclasses for AgentPlayer and ComputerPlayer. AgentPlayer used the Q-learning strategy developed in Part 2.1, while ComputerPlayer used the naïve strategy of always moving in the direction of the ball.

The original MDP had a state space of $(144)(2)(3)(12) + 1 = 10369$. With the addition of a second player, we had to add an additional state for when the ball went out of bounds for the opposite player, bringing the total state space up to 10370. We did not keep track of the position of the opponent's paddle, as it is not under the agent's control and should not affect its strategy in any way.

The actions and reward model were not changed from the original model described in Part 2.1.

The only negative consequence we noticed was that each game was shorter, as the computer player was very likely to miss the ball. Although this allowed us to run more training games in the same amount of time, it also meant that the Q-learning agent needed to play many more games to gain experience.

Hyperparameters

The hyperparameters were tuned in a similar way as in Part 2.1. We tried various values of $C_\alpha \in [250, 10000]$ and found an optimal value of $C_\alpha = 400$. We tried various values of $\gamma \in [0.1, 0.9]$ and found an optimal value of $\gamma = 0.5$. We tried various values of $C_\epsilon \in [100, 600]$ and found an optimal value of $C_\epsilon = 300$.

It is interesting to note that the optimal values of γ and C_ϵ were the same as for Part 2.1, but a larger C_α worked better for Part 2.2. This makes sense, as the average number of bounces in a multi-player game is much fewer since the computer player often misses the ball. Since the length of each game is shorter, the agent takes longer to learn an optimal strategy. A larger value of C_α results in a longer lasting learning rate α, allowing the agent to continue learning even after 10,000 games.

Performance

Using the aforementioned optimized hyperparameters, we attained the following results. With 100,000 training games, we were able to attain a win rate of 85.4% averaged over 1,000 testing games.

Part 2: Extra Credit

GUI

As briefly mentioned in Part 2.1, we implemented a GUI using the `pygame` framework. This greatly helped in debugging our Pong application. If you would like to watch the agent play, you can launch the GUI like so:

```
python pong --player2=agent --gui
```

Human player vs. AI

In addition to the AgentPlayer and ComputerPlayer subclasses, I added a third HumanPlayer subclass allowing a human to play against the AI. In fact, I generalized the implementation to the extent that two humans can play against each other, two computers can play against each other, two Q-learning agents can play against each other, and any other possible combination.

If you would like to challenge the AI yourself, you can run the application like so:

```
python pong --player1=human --player2=agent --gui
```

For human players, player 1 can be controlled by the W and S keys, while player 2 can be controlled by the up and down keys on the keyboard.

With only 10,000 training games playing by itself, the Q-learning agent becomes difficult to beat, although far from perfect. The agent exhibits impressive predictive capabilities, moving into position long before the ball arrives. However, the behavior of the agent's paddle is too jittery to confuse it with another human, so it can't pass the Turing test yet.

Since there is no variability in the starting velocities of the ball, the agent always uses the same opening move. Immediately after hitting the ball, the agent moves to the corner of the board where it expects the ball to rebound to. Since it was training alone, it thinks this will be where the ball will move next, but since it is now playing against a human, there is an additional bounce to randomize the velocities. Once it realizes this, it generally moves closer to the center of the board, then converges to the actual position where it thinks the ball will hit.

The agent doesn't really have any weaknesses that a human player can exploit, as the human player doesn't have any control over which direction the ball can bounce. An interesting addition to the game would be to make the bounce y-velocity a function of the position of the paddle at which the ball bounces. This would allow the human player (and of course the agent) to compete more directly, like in a real game of ping-pong.

Animation

We used QuickTime to record video of the AI agent playing the game. If you look in the zip file, `single-player.mov` is a video of the AI agent playing by itself after 10,000 training games. `computer-vs-agent.mov` is a video of the naïve computer playing against the agent. `human-vs-agent.mov` is a video of Adam holding his own against the AI agent in intense battle.

Added complexity

Markov Decision Processes are designed to make decisions in stochastic environments. However, our transition model is barely stochastic, as randomness is only incorporated into the ball velocity at each bounce. I found a way to fix this. Instead of adding randomness at each bounce, I added random values to the x and y velocities of the ball at every time-step. This results in a ball that can wiggle, curve, and fool a human opponent. Impressively enough, the AI agent is able to keep up fairly well. Without tuning any hyperparameters and with only 10,000 training games, the Q-learning agent is able to attain 8.706 bounces averaged over 1,000 testing games.

In the zip file, there is a `human-vs-agent-stochastic.mov` video showing Adam attempting to play against the AI under this stochastic transition model. As you can see, it becomes very difficult to predict where the ball will move next.

If you want to play against the AI in this stochastic environment, simply run:

```
python pong --player1=human --player2=agent --gui --stochastic
```

Statement of Individual Contribution

Abhinav solved Part 1 of the assignment, while Adam solved Part 2.